

Exercise 6

Express.js Basics — Beginner Lab Manual

What's inside

6(a)	Hello World message in the browser using a route
6(b)	A small website with multiple routes
6(c)	Print 'Hello World' in the browser console
6(d)	CRUD operations (Create, Read, Update, Delete)
6(e)	Connect the API to a MySQL database (mysql2 driver)

Before you start (do this once)

1. Install **Node.js** (LTS version) from nodejs.org.
2. Open a terminal and check it works: `node -v`
3. Create a project folder, open it in VS Code, and run the commands below.

TERMINAL

```
mkdir express-lab
cd express-lab
npm init -y          # creates package.json
npm install express  # downloads Express
```

Key words you will see

Word	Meaning (simple)
<code>express()</code>	Creates your web server app
<code>app.get(path, ...)</code>	Runs code when the browser opens that path (URL)
<code>req</code>	Request - what the browser sent to the server
<code>res</code>	Response - what the server sends back
<code>res.send()</code>	Sends text/HTML back to the browser
<code>app.listen(3000)</code>	Starts the server on port 3000

TIP: To stop a running server, press **Ctrl + C** in the terminal. After changing code, stop and run `node app.js` again.

Program 6(a): Hello World Route

Show a 'Hello World' message in the browser using Express

AIM: To write a program that displays the 'Hello World' message in the browser through a route using Express.js.

How it works



Program (app.js)

```
app.js

const express = require('express'); // import Express
const app = express();              // create the app

// Route: when someone opens "/", send Hello World
app.get('/', (req, res) => {
  res.send('Hello World');
});

// Start the server on port 3000
app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Run it

TERMINAL

```
node app.js
```

OUTPUT - TERMINAL

```
Server running at http://localhost:3000
```

OUTPUT - BROWSER

```
Browser at http://localhost:3000
Hello World
```

Result: The 'Hello World' message was displayed in the browser using an Express route.

Program 6(b): Website with Multiple Routes

Home, About and Contact pages

AIM: To develop a small website with multiple routes using Express.js.

Route map

URL in browser	Route in code	Page shown
localhost:3000/	app.get('/')	Home Page
localhost:3000/about	app.get('/about')	About Page
localhost:3000/contact	app.get('/contact')	Contact Page

Program (website.js)

website.js

```
const express = require('express');
const app = express();

// Menu links shown on every page
const menu = `
  <a href="/">Home</a> |
  <a href="/about">About</a> |
  <a href="/contact">Contact</a>
`;

app.get('/', (req, res) => {
  res.send(menu + '<h1>Home Page</h1><p>Welcome to our college website.</p>');
});

app.get('/about', (req, res) => {
  res.send(menu + '<h1>About Page</h1><p>We are B.Tech CSE students.</p>');
});

app.get('/contact', (req, res) => {
  res.send(menu + '<h1>Contact Page</h1><p>Email: college@example.com</p>');
});

app.listen(3000, () => {
  console.log('Website running at http://localhost:3000');
});
```

TERMINAL

```
node website.js
```

OUTPUT - BROWSER

Home | About | Contact

Home Page

Welcome to our college website.

(Click "About") -> About Page - We are B.Tech CSE students.

(Click "Contact") -> Contact Page - Email: college@example.com

Result: A small website with three routes (Home, About, Contact) was created using Express.js.

Program 6(c): Hello World in the Browser Console

Print a message inside the browser's Developer Tools console

AIM: To write a program to print 'Hello World' in the browser console using Express.js.

Two different consoles

Console	Where you see it	Who prints it
Terminal (server)	VS Code / Command Prompt	Node.js code: console.log()
Browser console	Press F12 > Console tab	<script> code sent to browser

Program (console.js)

console.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  // This <script> runs inside the browser
  res.send(`
    <h1>Press F12 and open the Console tab</h1>
    <script>
      console.log('Hello World');
    </script>
  `);
});

app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

TERMINAL

```
node console.js
```

OUTPUT - BROWSER

Page shows: Press F12 and open the Console tab

F12 > Console:
> Hello World

TIP: The backticks `` let us write HTML on many lines inside res.send().

Result: 'Hello World' was printed in the browser console using Express.js.

Program 6(d): CRUD Operations

Create, Read, Update and Delete students (stored in an array)

AIM: To implement CRUD operations using Express.js.

CRUD = 4 basic operations

Operation	HTTP Method	URL	What it does
Create	POST	/students	Add a new student
Read	GET	/students	Show all students
Update	PUT	/students/:id	Change a student's name
Delete	DELETE	/students/:id	Remove a student

Program (crud.js)

```
crud.js

const express = require('express');
const app = express();
app.use(express.json()); // lets us read JSON data sent by the client

// Our "database" - a simple array
let students = [
  { id: 1, name: 'Ravi' },
  { id: 2, name: 'Sita' }
];
let nextId = 3;

// READ - get all students
app.get('/students', (req, res) => {
  res.json(students);
});

// CREATE - add a new student
app.post('/students', (req, res) => {
  const student = { id: nextId++, name: req.body.name };
  students.push(student);
  res.json(student);
});

// UPDATE - change a student's name
app.put('/students/:id', (req, res) => {
  const id = Number(req.params.id);
  const student = students.find(s => s.id === id);
  if (!student) return res.status(404).send('Student not found');
  student.name = req.body.name;
  res.json(student);
});

// DELETE - remove a student
app.delete('/students/:id', (req, res) => {
  const id = Number(req.params.id);
  students = students.filter(s => s.id !== id);
  res.send('Student deleted');
});

app.listen(3000, () => console.log('CRUD API running at http://localhost:3000'));
```

How to test

A browser can only send GET requests. To test POST, PUT and DELETE, use **Postman** or the **Thunder Client** extension in VS Code. For POST and PUT, choose **Body > JSON**.

#	Method + URL	Body (JSON)	Response
1	GET /students	-	[{"id":1,"name":"Ravi"}, {"id":2,"name":"Sita"}]
2	POST /students	{"name":"Kiran"}	{"id":3,"name":"Kiran"}
3	PUT /students/1	{"name":"Ravi Kumar"}	{"id":1,"name":"Ravi Kumar"}

#	Method + URL	Body (JSON)	Response
4	DELETE /students/2	-	Student deleted
5	GET /students	-	[{"id":1,"name":"Ravi Kumar"}, {"id":3,"name":"Kiran"}]

NOTE: The data is stored in an array, so it resets every time you restart the server. Program 6(e) uses a real database to keep data permanently.

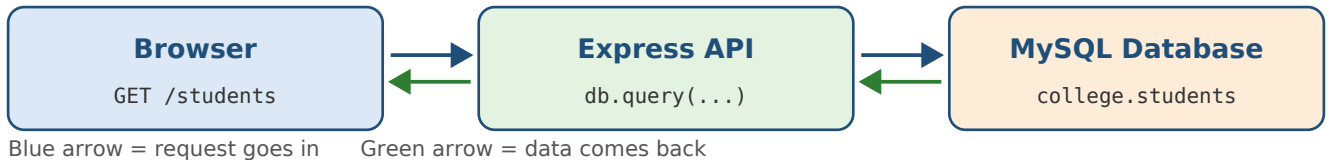
Result: CRUD operations were implemented successfully using Express.js.

Program 6(e): Connect API to MySQL

Using the mysql2 driver in Express

AIM: To establish a connection between an Express API and a MySQL database using the MySQL driver.

How it works



Step 1: Create the database (in MySQL Command Line Client)

MYSQL

```
CREATE DATABASE college;
USE college;

CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),
  dept VARCHAR(10)
);

INSERT INTO students (name, dept) VALUES ('Ravi', 'CSE'), ('Sita', 'ECE');
```

Step 2: Install the MySQL driver

TERMINAL

```
npm install mysql2
```

Step 3: Program (db.js)

db.js

```
const express = require('express');
const mysql = require('mysql2');
const app = express();

// Connection details - change the password to yours
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  database: 'college'
});

// Try to connect
db.connect((err) => {
  if (err) {
    console.log('Connection failed: ' + err.message);
    return;
  }
  console.log('Connected to MySQL database');
});

// API route: get all students from the database
app.get('/students', (req, res) => {
  db.query('SELECT * FROM students', (err, results) => {
    if (err) return res.status(500).send(err.message);
    res.json(results);
  });
});

app.listen(3000, () => console.log('API running at http://localhost:3000'));
```

TERMINAL

```
node db.js
```

OUTPUT - TERMINAL

```
API running at http://localhost:3000
Connected to MySQL database
```

OUTPUT - BROWSER

```
Browser at http://localhost:3000/students

[{"id":1,"name":"Ravi","dept":"CSE"}, {"id":2,"name":"Sita","dept":"ECE"}]
```

COMMON ERRORS: If you see **Access denied**, your password is wrong. If you see **Unknown database**, run Step 1 first.

Result: The Express API was connected to the MySQL database and student records were fetched successfully.